Mancalagent

Mahmut Zahid Goksu * Ömer İslam † Emre Duzakin ‡

April 15, 2026

1 Problem definition

Mangala is a traditional Turkish strategy game that dates back centuries and is part of the larger family of Mancala-style games played around the world. It is a two-player game involving a playing board with small pits and stones, where players take turns distributing the stones strategically to capture their opponent's pieces and control the board. Known for its simple rules yet deep tactical possibilities, Mangala has been a popular pastime in Turkish culture in the Ottoman era, often associated with intelligence and strategic thinking. Mangala consists of 12 small pits, 6 for each player and 2 treasuries as illustrated in Fig.1. Each player aims to capture as many stones as possible by capturing stones to their own treasuries.

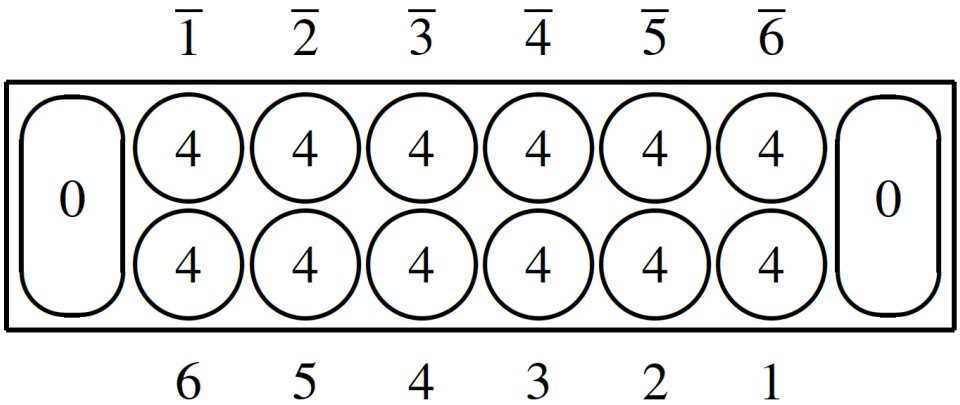

Figure 1: Illustration of a Mangala board. Each player has six small pits, each initially containing four stones. Additionally, each player has a treasury (also known as a store) located to their right, where captured stones are collected.

Kalah a similar game to our Mangala has been shown to be solved in [1]. Kalah has been shown to have a state-space complexity of 1.31×10^{13}, which can be considered a relatively large search space. [1] has shown that when played optimally, the first player can win by at least 3 stones. In this project, we aim to train a Reinforcement Learning (RL) agent using two main algorithms—Monte Carlo Tree Search and TD-Gammon—with the goal of emulating optimal play that can consistently win against average players.

*NO: 79520 Email: mgoksu21@ku.edu.tr
†NO: 79909 Email: oislam21@ku.edu.tr
‡NO: 69663 Email: eduzakin18@ku.edu.tr

2 The current RL formulation of the problem

- **State:** Due to the fully observable nature of the game, our states representation is an array of the board along with the action taken within the state. In our implementation search board state is represented as a vector see in 1, where the first 5 indexes represent current players pits and P1 represents the store.

$$[1, 2, 3, 4, 5, 6, P1, 1', 2', 3', 4', 5', 6', P2]^T \tag{1}$$

- **Action:** As illustrated in Fig.1 the action space of an agent is any one of the pits on a players' respective side marked from 1-6.

- **Rewards:** Due to the nature of the game our rewards are sparse and are received in Win, lose and draw conditions for MCST algorithm, where as for the TD-Gammon rewards are received at each player move. additional to these rewards there are several different dynamics that Mangala offers such as capturing stones, which can happen in several conditions. Current rewards system only looks at the difference occurred in the player pits however, these dynamics could be incorporated in the rewards to help the model perform better in earlier steps.

3 Solution algorithms

we will be employing two different solutions to Mangala. The first of these is the MCTS algorithm. In addition we will be implementing the TD-Gammon algorithm

3.1 RL with Monte Carlo Tree Search

Monte Carlo Tree Search is an iterative algorithm that searches the state space and builds statistical evidence on the decisions available in particular states [3]. MCTS have been utilized in game playing in the past with AlphaGO [2], where they use a MCTS algorithm with pre-trained Fast-fast roll-out and supervised value network. Due to limited resources and data we were not able to train a fast-rollout and superviesed vlaue network but we have gotten the inspiration from AlphaGO. MCST consists of four step: Selection , Expantion, Simpulation and Backpropagation. In our implementation:

3.1.1 Selection

For our selection strategy we directly use the strategy used in AlphaGO. For selection we use the formula 4, where $Q(s_t, a)$ is given by 3 and $U(s_t, a)$ is given by 2. $P(s, a)$ represents the probability of an state and action. $P(s, t)$ is initially uniform for all possible actions and after the first iteration of our tree will be calculated using the Policy network REINFORCE and $N(s_t, a)$ represents the visit count of the edge $e(s, a)$

$$U(s, a) = c_{\text{puct}}\, P(s, a)\, \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)} \tag{2}$$

$$Q(s, a) = \frac{W(s, a)}{N(s, a)} \tag{3}$$

$$a_t = \arg\max_a \big(Q(s_t, a) + U(s_t, a)\big) \tag{4}$$

3.1.2 Expansion

For our expansions strategy we use a every search, every child expansion strategy. From this we mean that with each search, when encountered a leaf node of the built tree we expand each possible child of that leaf node. This allows us to build the tree faster due to the limited amount of moves possible (maximum possible action is 6) while also accumulating visits and values of each edge.

3.1.3 Simulation

For our simulation strategy we use a random roll-out due to the absence of labeled data to train a fast roll-out network.

3.1.4 Backpropagation

After receiving rewards from our simulations, we propagate back until the root node, incrementing visit count and values of edges in the path from the root to the expanded leaf node.

3.1.5 Training Policy and Value Network

The MCTS algorithm enforces a policy π_{mcst} with each iteration, and the resulting transitions can be used to train our policy and Value networks. We create two different networks NN_{policy} and NN_{value}. we use a simple linear layer followed by nonlinear transformation.

In addition to training these networks we later want to use these to rebuild the MCTS tree to get even better result in using the NN_{policy} network for prior probabilities for edges and NN_{value} network for evaluating return given by formula 5 where z_t is the return from random roll-out.

$$V(s_t, a) = \lambda NN_{value}(s_t) - (1 - \lambda) z_t \tag{5}$$

3.2 TD-Gammon algorithm

TD-Gammon is an derivation of the TD-Gamma algorithm, which is an online learning algorithm that updates the state values based on eligibility traces. TD-Gammon builds on top of the regular TD-Gamma idea by incorporating a deep network to approximate the value of the state rather than keeping track of the states in a tabular manner[4].

The eligibility traces are stored in a look-up table for each episodes and updated based on the value approximation of the states by the network. As for the network update, the based on the current state and the eligibility vector every parameter is updated for each move taken by a player.

Since the TD-Gammon algorithm used to create a network that aims to approximate the state values, algorithm is applied independent of the turn players. Additionally, to promote exploration for the initial states, ϵ-greedy algorithm with a decreasing epsilon based on the ϵ is used.

$$\delta_k = r_{k+1} + \gamma V(s_{k+1}, w) - V(s_k, w) \tag{6}$$

$$\Delta w_t = \alpha \sum_{k=t}^{T-1} \nabla_w V(s_k, w) \cdot \delta_k \cdot \lambda^{k-t} \tag{7}$$

4 Current Progress and Future work

We have implemented the game using python and successfully finished all test required for the game. in addition we have implemented our first iterations of our Reinforcement learning algorithms. we have not used any libraries other than PyTorch to train our models and build the data structures needed.

currently we are working on refining our code, for ease of use, and training our algorithms to enhance the performance of our models in playing Mangala. we plan to train multiple agents and let them play against each other. In addition to naively letting the models play the games we will also be implementing a planning algorithm. This will use our pre-trained models in addition to planning.

5 Results

Fig.2 shows the training plots of losses from our RL MCTS search algorithm. 10000 iterations are used to get build the tree and get samples. Using the samples from these iterations train a the NN_{policy} and NN_{value} networks

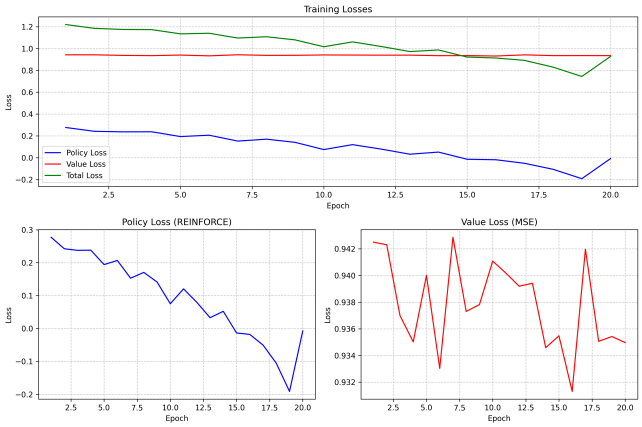

Figure 2: Training of value and policy networks using MCTS self-play dataset (We might have some bug in our code, these plots my not show real results)

As for the TD-Gammon results, while the initial loss graph for 1000 games based on the TD-error seems to be converging with high variance present. While the loss is still high it is an improvement over our initial implementation which contained NaN values due to the exploding gradients resulted by the use of ReLU in the value network.

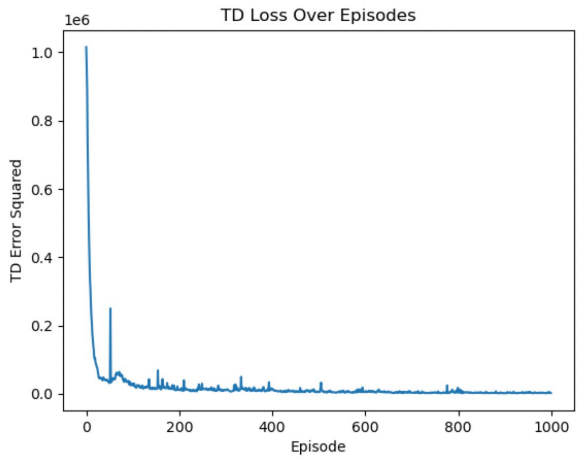

Figure 3: Training losses of TD-Gammon with ϵ-greedy exploration

6 Implementation Information

All of the code for our project is written in python which is composed of the game simulation, agent implementation, training and testing/inference. Currently we have a working game simulation that saves every game state of a started game to a history file to be used in training phase.

For our agents, we have created a base agent with the expected functionalities such as given a state returning a selected action,for our case a player pit. We have created several agents that extend the base agent such as human agent for collecting sample game with real players, as well as a random agent that acts randomly to collect many game samples in a short time.

All code for of our progress can be found in our GitHub repository `https://github.com/Mancalagent/Mangalagent`

References

[1] Geoffrey Irving, Jeroen Donkers, and Jos Uiterwijk. Solving kalah. *ICGA journal*, 23, 05 2003.

[2] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy P. Lillicrap, Karen Simonyan, and Demis Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *CoRR*, abs/1712.01815, 2017.

[3] Maciej Swiechowski, Konrad Godlewski, Bartosz Sawicki, and Jacek Mandziuk. Monte carlo tree search: A review of recent modifications and applications. *CoRR*, abs/2103.04931, 2021.

[4] Gerald Tesauro. Temporal difference learning and td-gammon. *Commun. ACM*, 38(3):58–68, March 1995.

7